

第六讲 并行程序设计基础

何克晶

kejinghe@gmail.com

华南理工大学

计算机科学与工程学院

目录

并行算法的设计例子：矩阵乘法

Cannon

DNS

并行程序设计概述

并行程序设计的基本问题

并行程序设计模型

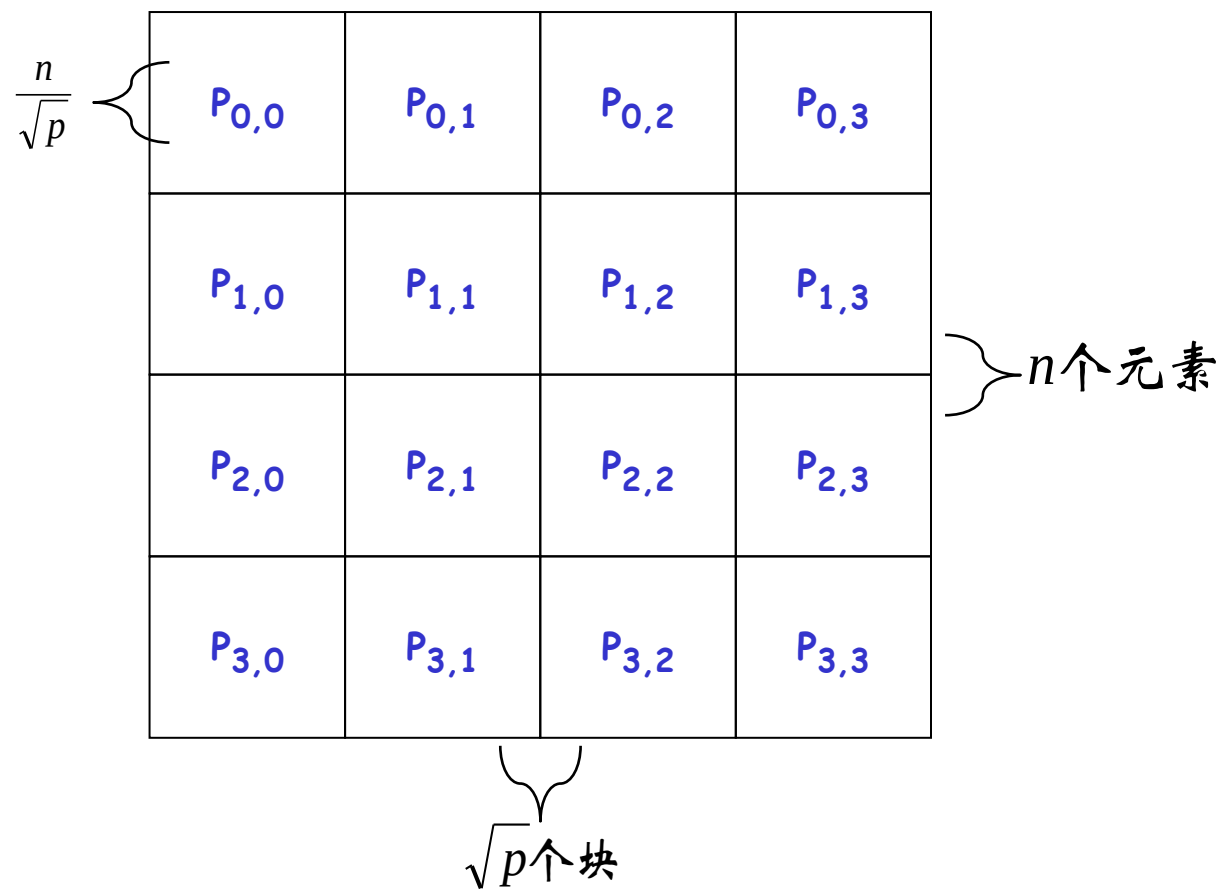


矩阵乘法

- 普通串行算法（算法9.3）的运行时间为 $O(n^3)$
- 已知串行算法时间复杂度为 $O(n^x)$, $2 < x < 3$
- Matrix multiplication is the most studied parallel algorithm.
- It is a good algorithm to learn because it shows many ideas about parallelism.
- Parallel algorithms:
 - 简单分块算法
 - Cannons
 - DNS

矩阵分块

- 分块: A 、 B 和 C 分成 $p = \sqrt{p} \times \sqrt{p}$ 的方块阵 $A_{i,j}$ 、 $B_{i,j}$ 和 $C_{i,j}$, 大小均为 $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}}$
- p 个处理器编号为 $(P_{0,0}, \dots, P_{0,\sqrt{p}-1}, \dots, P_{\sqrt{p}-1,\sqrt{p}-1})$ 。 $P_{i,j}$ 存放 $A_{i,j}$ 、 $B_{i,j}$ 和 $C_{i,j}$ ($n \gg p$)



简单并行分块算法

- $P_{i,j}$ 存放 $A_{i,j}$ 、 $B_{i,j}$ 和 $C_{i,j}$ ($n \gg p$)
- 算法:
 - ①通信
 - 每行处理器进行 A 矩阵块的多到多播送 (得到 $A_{i,k}$, $k=0 \sim \sqrt{p}-1$)
 - 每列处理器进行 B 矩阵块的多到多播送 (得到 $B_{k,j}$, $k=0 \sim \sqrt{p}-1$)
 - ②乘-加运算: $P_{i,j}$ 做 $C_{i,j} = \sum_{k=0}^{\sqrt{p}-1} A_{ik} \cdot B_{kj}$
- 计算时间?
- 存在的问题?

矩阵分块的颜色表示

每个三角代表一个矩阵块

只有相同颜色的三角可以相乘



块的重排



块 A_{ij} 向左循环
移动 i 步

块 B_{ij} 向上循环
移动 j 步

Cannon算法描述

- 算法:

- ① 对准:

- 所有块 $A_{i,j}$ ($0 \leq i, j \leq \sqrt{p} - 1$) 向左循环移动 i 步;

- 所有块 $B_{i,j}$ ($0 \leq i, j \leq \sqrt{p} - 1$) 向上循环移动 j 步;

- ② 所有处理器 $P_{i,j}$ 做执行 $A_{i,j}$ 和 $B_{i,j}$ 的乘-加运算;

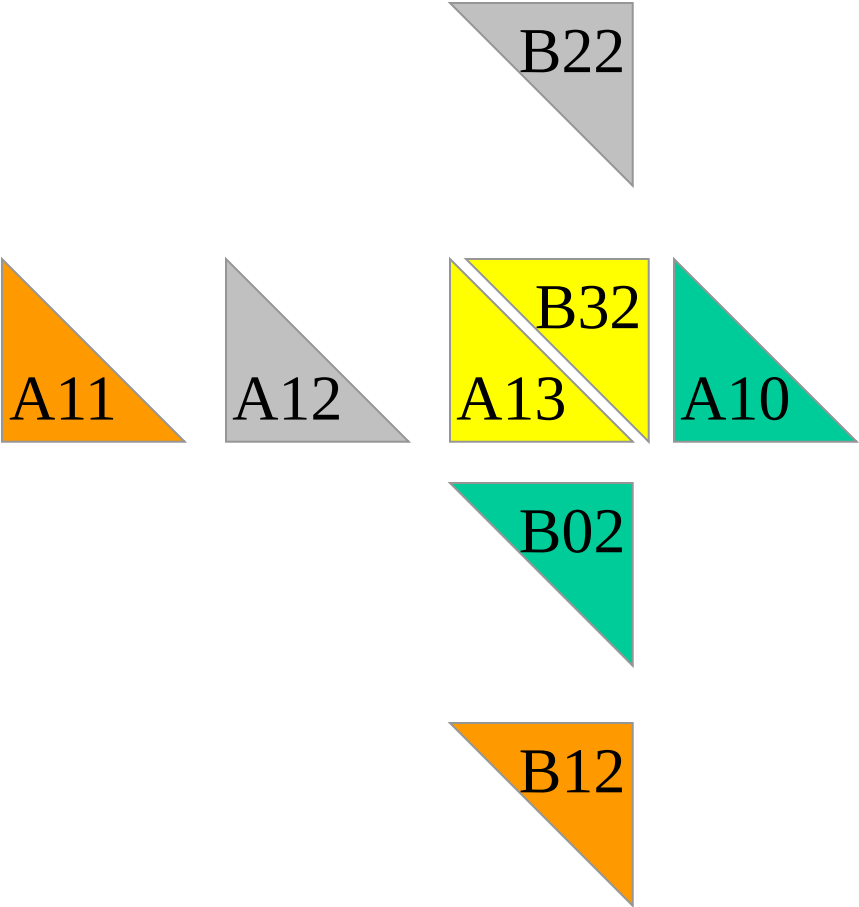
- ③ 移位:

- A 的每个块向左循环移动一步;

- B 的每个块向上循环移动一步;

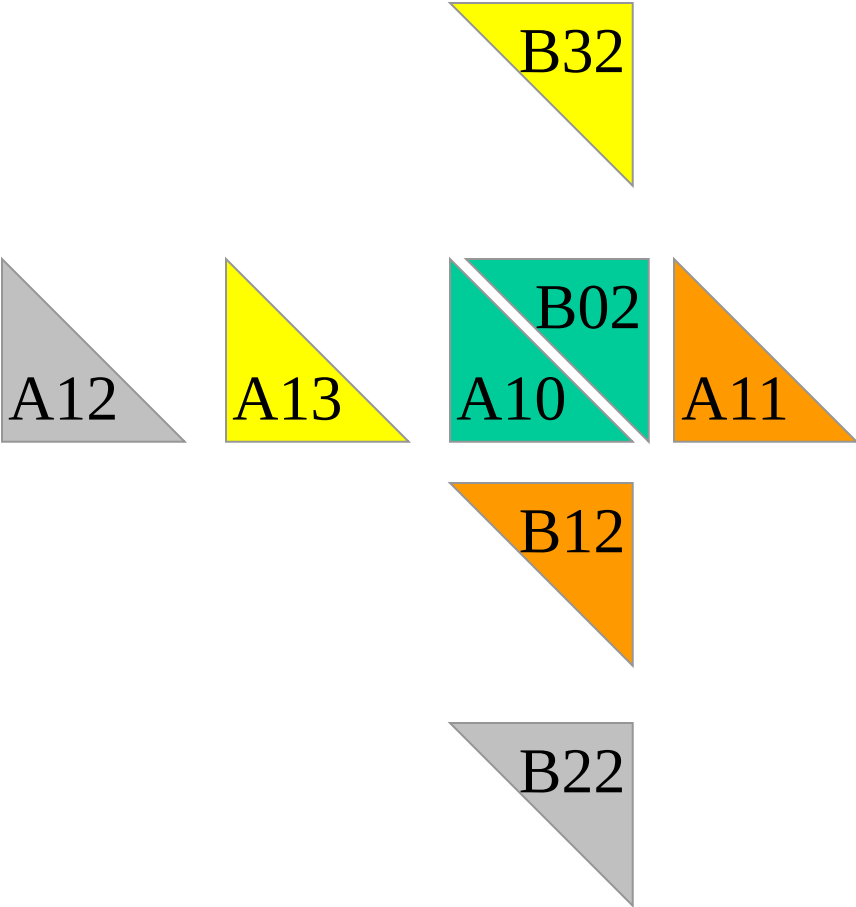
- ④ 转② 执行 $\sqrt{p} - 1$ 次;

考虑处理器 $P_{1,2}$



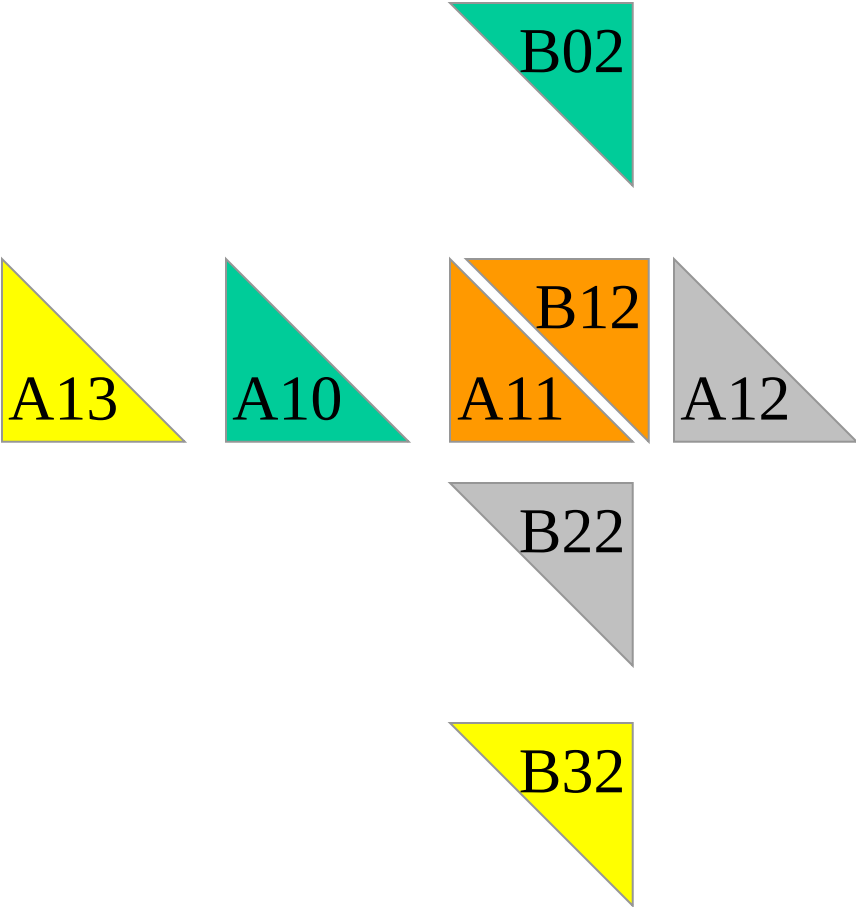
Step 1: 起始后对准

考虑处理器 $P_{1,2}$



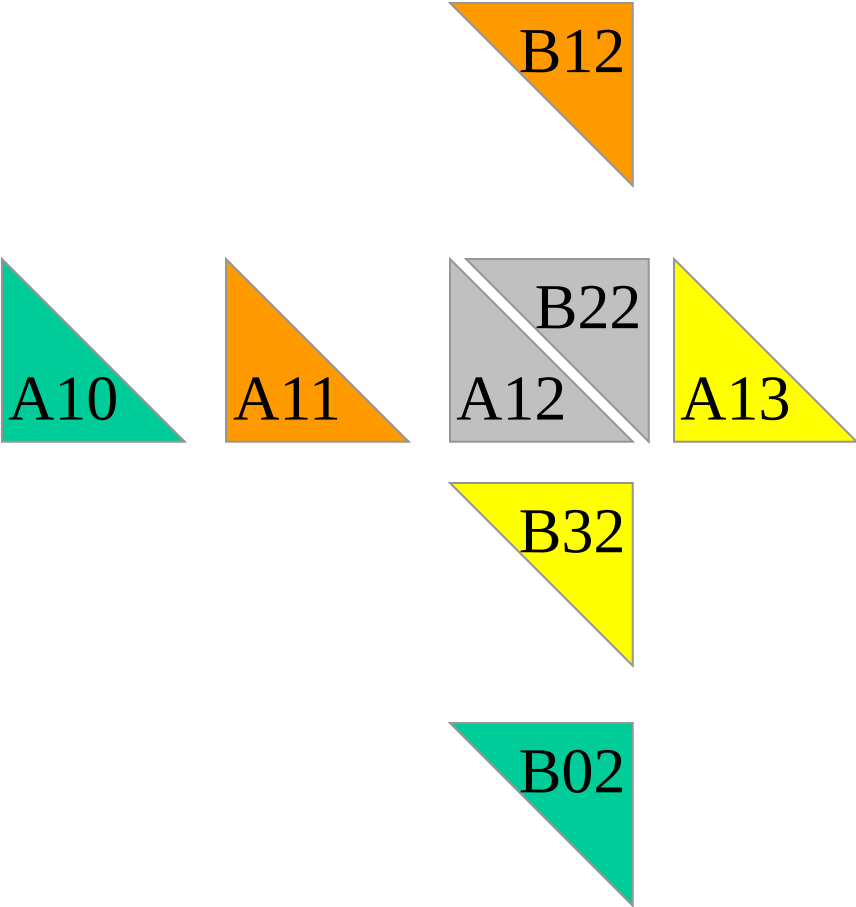
Step 2: 第一次移位后

考虑处理器 $P_{1,2}$



Step 3: 第二次移位后

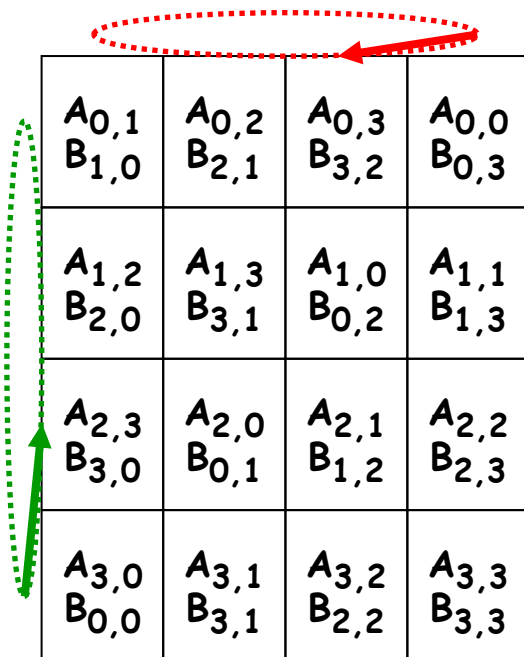
考虑处理器 $P_{1,2}$



Step 4: 第三次移位后

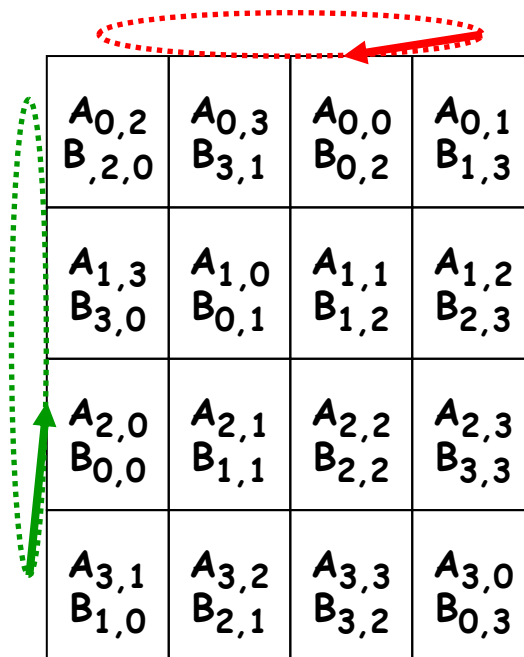
移位的结果

第一次移位后



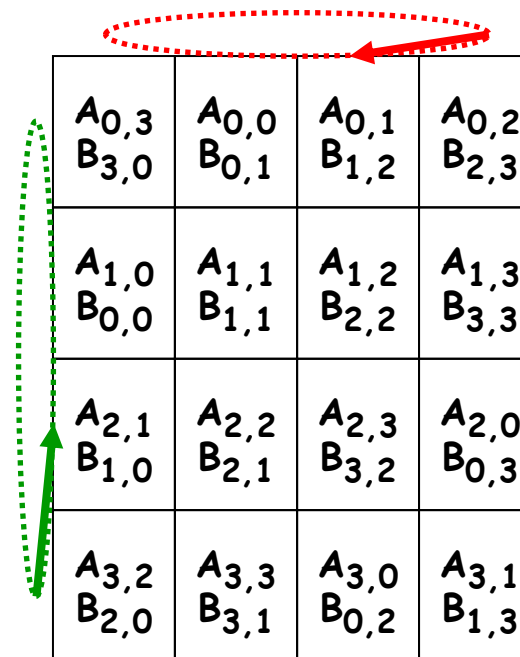
$A_{0,1}$ $B_{1,0}$	$A_{0,2}$ $B_{2,1}$	$A_{0,3}$ $B_{3,2}$	$A_{0,0}$ $B_{0,3}$
$A_{1,2}$ $B_{2,0}$	$A_{1,3}$ $B_{3,1}$	$A_{1,0}$ $B_{0,2}$	$A_{1,1}$ $B_{1,3}$
$A_{2,3}$ $B_{3,0}$	$A_{2,0}$ $B_{0,1}$	$A_{2,1}$ $B_{1,2}$	$A_{2,2}$ $B_{2,3}$
$A_{3,0}$ $B_{0,0}$	$A_{3,1}$ $B_{3,1}$	$A_{3,2}$ $B_{2,2}$	$A_{3,3}$ $B_{3,3}$

第二次移位后



$A_{0,2}$ $B_{2,0}$	$A_{0,3}$ $B_{3,1}$	$A_{0,0}$ $B_{0,2}$	$A_{0,1}$ $B_{1,3}$
$A_{1,3}$ $B_{3,0}$	$A_{1,0}$ $B_{0,1}$	$A_{1,1}$ $B_{1,2}$	$A_{1,2}$ $B_{2,3}$
$A_{2,0}$ $B_{0,0}$	$A_{2,1}$ $B_{1,1}$	$A_{2,2}$ $B_{2,2}$	$A_{2,3}$ $B_{3,3}$
$A_{3,1}$ $B_{1,0}$	$A_{3,2}$ $B_{2,1}$	$A_{3,3}$ $B_{3,2}$	$A_{3,0}$ $B_{0,3}$

第三次移位后



$A_{0,3}$ $B_{3,0}$	$A_{0,0}$ $B_{0,1}$	$A_{0,1}$ $B_{1,2}$	$A_{0,2}$ $B_{2,3}$
$A_{1,0}$ $B_{0,0}$	$A_{1,1}$ $B_{1,1}$	$A_{1,2}$ $B_{2,2}$	$A_{1,3}$ $B_{3,3}$
$A_{2,1}$ $B_{1,0}$	$A_{2,2}$ $B_{2,1}$	$A_{2,3}$ $B_{3,2}$	$A_{2,0}$ $B_{0,3}$
$A_{3,2}$ $B_{2,0}$	$A_{3,3}$ $B_{3,1}$	$A_{3,0}$ $B_{0,2}$	$A_{3,1}$ $B_{1,3}$

Cannon分块乘法

//输入: $A_{n \times n}$, $B_{n \times n}$; 输出: $C_{n \times n}$

Begin

```
(1) for k=0 to  $\sqrt{p} - 1$  do
  for all  $P_{i,j}$  par-do
    (i) if  $i > k$  then
       $A_{i,j} \leftarrow A_{i,(j+1) \bmod \sqrt{p}}$ 
    endif
    (ii) if  $j > k$  then
       $B_{i,j} \leftarrow B_{(i+1) \bmod \sqrt{p}, j}$ 
    endif
  endfor
endfor
```

算法9.5

```
(2) for all  $P_{i,j}$  par-do  $C_{i,j} = 0$  endfor
(3) for k=0 to  $\sqrt{p} - 1$  do
  for all  $P_{i,j}$  par-do
    (i)  $C_{i,j} = C_{i,j} + A_{i,j} B_{i,j}$ 
    (ii)  $A_{i,j} \leftarrow A_{i,(j+1) \bmod \sqrt{p}}$ 
    (iii)  $B_{i,j} \leftarrow B_{(i+1) \bmod \sqrt{p}, j}$ 
  endfor
endfor
End
```

复杂度分析

- 算法有 $\sqrt{p}$ 次循环
- 在每次循环, 有 $(n/\sqrt{p}) \times (n/\sqrt{p})$ 的矩阵乘法: $\Theta(n^3/p^{3/2})$
- 计算复杂度: $\Theta(n^3/p)$
- 在每个循环, 每个处理器发送和接收两个大小为 $(n/\sqrt{p}) \times (n/\sqrt{p})$ 的数据块
 - 每个处理器的通信复杂度: $\Theta(n^2/\sqrt{p})$
- 串行算法: $\Theta(n^3)$
- 并行开销: $\Theta(n^2\sqrt{p})$

目录

并行算法的设计例子：矩阵乘法

Cannon

DNS

并行程序设计概述

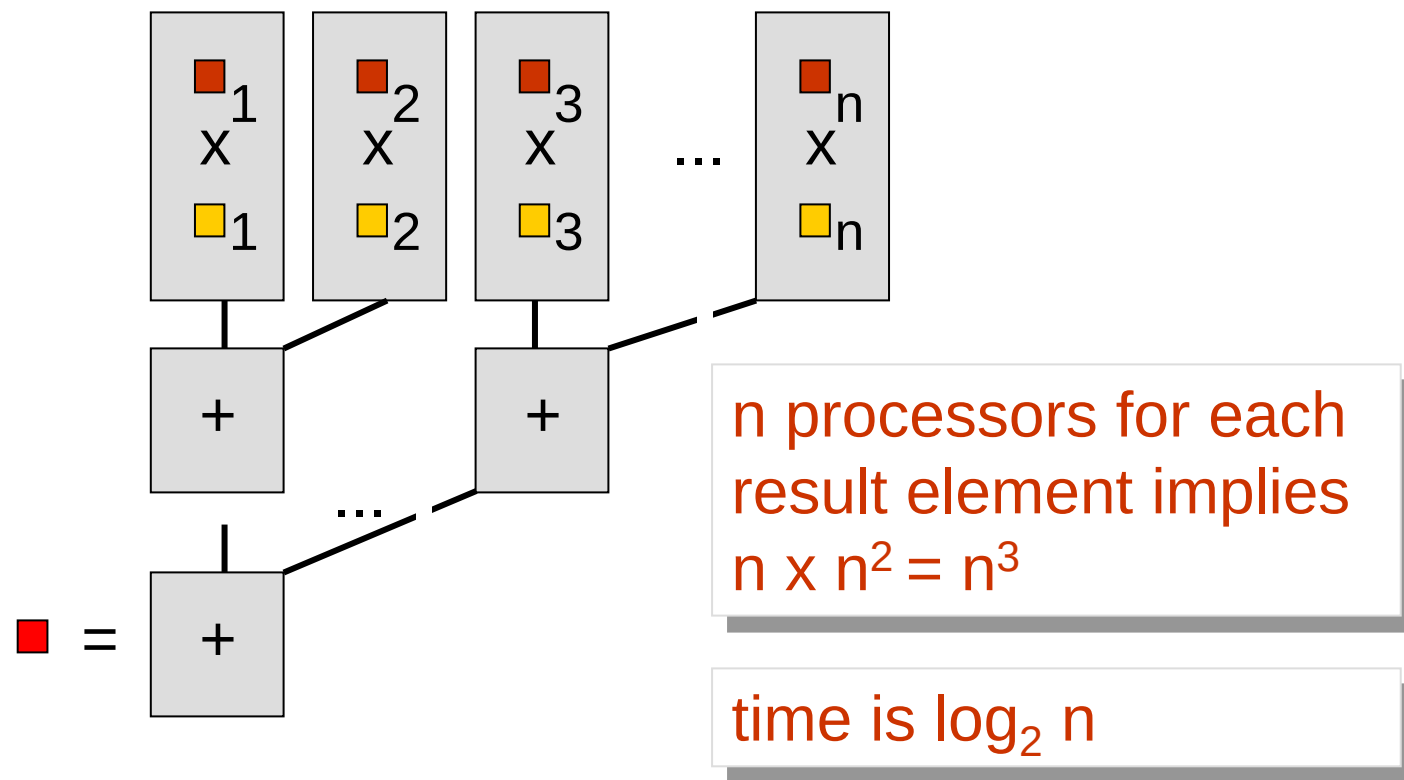
并行程序设计的基本问题

并行程序设计模型



DNS矩阵乘法的思路

- Motivation: From a good and common idea



是否可利用更多的处理器达到更高的加速比?

DNS矩阵分块

- 背景: 由Dekel、Nassimi和Sahni（1981）提出的SIMD-CC上的矩阵乘法, 处理器数目为 n^3 , 运行时间为 $O(\log n)$, 是一种速度很快的算法
- 基本思想: 通过一到一和一到多的播送办法, 使得处理器 (k,i,j) 拥有 $a_{i,k}$ 和 $b_{k,j}$, 进行本地相乘, 再沿 k 方向进行单点积累求和, 结果存储在处理器 $(0,i,j)$ 中
- 处理器编号: 处理器数 $p=n^3= (2^q)^3=2^{3q}$, 处理器 P_r 位于位置 (k,i,j) , 这里 $r=kn^2+in+j$, $(0 \leq i, j, k \leq n-1)$ 。位于 (k,i,j) 的处理器 P_r 的三个寄存器 A_r, B_r, C_r 分别表示为 $A[k,i,j]$, $B[k,i,j]$ 和 $C[k,i,j]$, 初始时均为0